| | |
|---|---|
| **Status** | Finished |
| **Started** | Thursday, 8 October 2026, 12:45 PM |
| **Completed** | Thursday, 8 October 2026, 1:00 PM |
| **Duration** | 15 mins 10 secs |
| **Grade** | **10.00** out of 10.00 (**100**%) |

Question **1**

Correct

Mark 1.00 out of 1.00

Given an array of integers `nums` which is sorted in ascending order, and an integer `target`, write a function to search `target` in `nums`. If `target` exists, then return its index. Otherwise, return `-1`.

You must write an algorithm with `O(log n)` runtime complexity.

**Example 1:**

```
Input: nums = [-1,0,3,5,9,12], target = 9
Output: 4
Explanation: 9 exists in nums and its index is 4
```

**Example 2:**

```
Input: nums = [-1,0,3,5,9,12], target = 2
Output: -1
Explanation: 2 does not exist in nums so return -1
```

**Constraints:**

- `1 <= nums.length <= 10^4`
- `-10^4 < nums[i], target < 10^4`
- All the integers in `nums` are **unique**.
- `nums` is sorted in ascending order.

**For example:**

| Test | Result |
|---|---|
| `print(search([-1,0,3,5,9,12],9))` | 4 |

**Answer:** (penalty regime: 0 %)

Reset answer

```
def search(nums: list[int], target: int) -> int:
    left, right = 0, len(nums) - 1

    while left <= right:
        mid = left + (right - left) // 2

        if nums[mid] == target:
            return mid
        elif nums[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1

```

| | Test | Expected | Got | |
|---|---|---|---|---|
| ✓ | `print(search([-1,0,3,5,9,12],9))` | 4 | 4 | ✓ |
| ✓ | `print(search([-1,0,3,5,9,12],2))` | -1 | -1 | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

String should contain only the words are not palindrome.

**Sample Input 1**

Malayalam is my mother tongue

**Sample Output 1**

is my mother tongue

**Answer:** (penalty regime: 0 %)

```
s = input().lower()
print(' '.join(w for w in s.split() if w != w[::-1]))
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | Malayalam is my mother tongue | is my mother tongue | is my mother tongue | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Given two Strings s1 and s2, remove all the characters from s1 which is present in s2.

**Constraints**

1<= string length <= 200

**Sample Input 1**

experience
enc

**Sample Output 1**

xpri

**Answer:** (penalty regime: 0 %)

```
def remove_chars(s1: str, s2: str) -> str:
    # Convert s2 into a set for O(1) lookups
    chars_to_remove = set(s2)

    # Keep characters from s1 only if they are not in the set
    result = [char for char in s1 if char not in chars_to_remove]

    # Join the characters back into a single string
    return "".join(result)

# Reading input dynamically for competitive programming platforms
import sys

if __name__ == "__main__":
    # Read all inputs from standard input
    input_data = sys.stdin.read().splitlines()

    # Ensure we have both strings to process
    if len(input_data) >= 2:
        s1 = input_data[0]
        s2 = input_data[1]
        print(remove_chars(s1, s2))

```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | experience<br>enc | xpri | xpri | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

Two string values S1, S2 are passed as the input. The program must print first N characters present in S1 which are also present in S2.

**Input Format:**

The first line contains S1.
The second line contains S2.
The third line contains N.

**Output Format:**

The first line contains the N characters present in S1 which are also present in S2.

**Boundary Conditions:**

2 <= N <= 10
2 <= Length of S1, S2 <= 1000

**Example Input/Output 1:**

Input:

abcbde
cdefghbb
3

Output:

bcd

**Note:**

b occurs twice in common but must be printed only once.

**Answer:** (penalty regime: 0 %)

```
import sys

def find_common_characters():
    # Read all lines from standard input
    input_data = sys.stdin.read().splitlines()

    if len(input_data) < 3:
        return

    s1 = input_data[0]
    s2 = input_data[1]
    n = int(input_data[2])

    # Convert s2 to a set for fast O(1) lookups
    s2_chars = set(s2)

    # To store the unique common characters and keep track of duplicates
    common_chars = []
    seen = set()

    # Iterate through s1 to preserve its character order
```

```
    for char in s1:
        if char in s2_chars and char not in seen:
            seen.add(char)
            common_chars.append(char)

            # If we have collected N characters, we can stop early
            if len(common_chars) == n:
                break

    # Join and print the first N common characters
    print("".join(common_chars))

if __name__ == "__main__":
    find_common_characters()

```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | abcbde<br>cdefghbb<br>3 | bcd | bcd | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Given an array `nums` containing `n` distinct numbers in the range `[0, n]`, return *the only number in the range that is missing from the array.*

**Example 1:**

```
Input: nums = [3,0,1]
Output: 2
Explanation: n = 3 since there are 3 numbers, so all numbers are in the range [0,3]. 2 is the missing number in the range since it does not appear in nums.
```

**Example 2:**

```
Input: nums = [0,1]
Output: 2
Explanation: n = 2 since there are 2 numbers, so all numbers are in the range [0,2]. 2 is the missing number in the range since it does not appear in nums.
```

**Example 3:**

```
Input: nums = [9,6,4,2,3,5,7,0,1]
Output: 8
Explanation: n = 9 since there are 9 numbers, so all numbers are in the range [0,9]. 8 is the missing number in the range since it does not appear in nums.
```

**For example:**

| Test | Result |
|---|---|
| print(missingNumber([3,0,1])) | 2 |
| print(missingNumber([0,1])) | 2 |

**Answer:** (penalty regime: 0 %)

Reset answer

```python
def missingNumber(nums: list[int]) -> int:
    missing = len(nums)

    # XOR all indices and all values together
    for i, num in enumerate(nums):
        missing ^= i ^ num

    return missing

```

| | Test | Expected | Got | |
|---|---|---|---|---|
| ✓ | print(missingNumber([3,0,1])) | 2 | 2 | ✓ |
| ✓ | print(missingNumber([0,1])) | 2 | 2 | ✓ |
| ✓ | print(missingNumber([9,6,4,2,3,5,7,0,1])) | 8 | 8 | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

An list contains N numbers and you want to determine whether two of the numbers sum to a given number K. For example, if the input is 8, 4, 1, 6 and K is 10, the answer is yes (4 and 6). A number may be used twice.

**Input Format**

The first line contains a single integer n , the length of list

The second line contains n space-separated integers, list[i].

The third line contains integer k.

**Output Format**

Print Yes or No.

**Sample Input**

7

0 1 2 4 6 5 3

1

**Sample Output**

Yes

**For example:**

| Input | Result |
|---|---|
| 5<br>8 9 12 15 3<br>11 | Yes |
| 6<br>2 9 21 32 43 43 1<br>4 | No |

**Answer:** (penalty regime: 0 %)

```
import sys

def two_sum_check():
    # Read all lines from standard input
    input_lines = sys.stdin.read().splitlines()

    if len(input_lines) < 3:
        return

    # Parse inputs cleanly
    n = int(input_lines[0].strip())
    nums = list(map(int, input_lines[1].strip().split()))
    k = int(input_lines[2].strip())

    # Store the frequency of each number using a dictionary
    num_counts = {}
    for num in nums:
        num_counts[num] = num_counts.get(num, 0) + 1

    # Validate pairs
    for num in nums:
        complement = k - num

        if complement in num_counts:
            # If the complement is the same number, we need at least 2 of them
            if complement == num and num_counts[num] < 2:
                continue

            print("Yes")
            return
```

```
31
32      print("No")
33
34  if __name__ == "__main__":
35      two_sum_check()
36
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | 5<br>8 9 12 15 3<br>11 | Yes | Yes | ✓ |
| ✓ | 6<br>2 9 21 32 43 43 1<br>4 | No | No | ✓ |
| ✓ | 6<br>13 42 31 4 8 9<br>17 | Yes | Yes | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Write a Python program for binary search.

**For example:**

| Input | Result |
|---|---|
| 1,2,3,5,8<br>6 | False |
| 3,5,9,45,42<br>42 | True |

**Answer:** (penalty regime: 0 %)

```
import sys

def binary_search(nums: list[int], target: int) -> bool:
    left, right = 0, len(nums) - 1

    while left <= right:
        mid = left + (right - left) // 2

        if nums[mid] == target:
            return True
        elif nums[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return False

def main():
    # Read all inputs from standard input
    input_lines = sys.stdin.read().splitlines()
    if len(input_lines) < 2:
        return

    # Parse comma-separated integers for the list
    nums = [int(x) for x in input_lines[0].strip().split(',')]
    # Parse the target integer
    target = int(input_lines[1].strip())

    # Binary search requires a sorted list
    nums.sort()

    # Perform search and print True/False
    result = binary_search(nums, target)
    print(result)

if __name__ == "__main__":
    main()

```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | 1,2,3,5,8<br>6 | False | False | ✓ |
| ✓ | 3,5,9,45,42<br>42 | True | True | ✓ |
| ✓ | 52,45,89,43,11<br>11 | True | True | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

Given an list, find peak element in it. A peak element is an element that is greater than its neighbors.

An element a[i] is a peak element if

A[i-1] <= A[i] >=a[i+1] for middle elements. [0<i<n-1]

A[i-1] <= A[i] for last element [i=n-1]

A[i]>=A[i+1] for first element [i=0]

**Input Format**

The first line contains a single integer n , the length of A .
The second line contains n space-separated integers,A[i].

**Output Format**

**Print** peak numbers separated by space.

**Sample Input**

5

8 9 10 2 6

**Sample Output**

```
10 6
```

**For example:**

| Input | Result |
|---|---|
| 4<br>12 3 6 8 | 12 8 |

**Answer:** (penalty regime: 0 %)

```
import sys

def find_peaks():
    # Read all lines from standard input
    input_lines = sys.stdin.read().splitlines()

    if len(input_lines) < 2:
        return

    # Parse length and the array of numbers
    n = int(input_lines[0].strip())
    # Handle empty array edge case
    if n == 0:
        return

    nums = list(map(int, input_lines[1].strip().split()))

    peaks = []

    # If there is only 1 element, it is automatically a peak
    if n == 1:
        peaks.append(nums[0])
    else:
        for i in range(n):
            # Condition for the first element
            if i == 0:
                if nums[0] >= nums[1]:
                    peaks.append(nums[0])
            # Condition for the last element
            elif i == n - 1:
                if nums[n - 1] >= nums[n - 2]:
                    peaks.append(nums[n - 1])
            # Condition for middle elements
            else:
                if nums[i - 1] <= nums[i] >= nums[i + 1]:
```

```
                    peaks.append(nums[i])

    # Print peak numbers separated by space
    print(*(peaks))

if __name__ == "__main__":
    find_peaks()

```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | 7<br>15 7 10 8 9 4 6 | 15 10 9 6 | 15 10 9 6 | ✓ |
| ✓ | 4<br>12 3 6 8 | 12 8 | 12 8 | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Balanced strings are those that have an equal quantity of 'L' and 'R' characters.

Given a balanced string s, split it in the maximum amount of balanced strings.

Return the maximum amount of split balanced strings.

Example 1:

Input:

RLRRLLRLRL

Output:

4

Explanation: s can be split into "RL", "RRLL", "RL", "RL", each substring contains same number of 'L' and 'R'.

Example 2:

Input:

RLLLLRRRLR

Output:

3

Explanation: s can be split into "RL", "LLLRRR", "LR", each substring contains same number of 'L' and 'R'.

Example 3:

Input:

LLLLRRRR

Output:

1

Explanation: s can be split into "LLLLRRRR".

Constraints:

1 <= s.length <= 1000

s[i] is either 'L' or 'R'.

s is a balanced string.

**For example:**

| Test | Result |
|---|---|
| `print(BalancedStrings('RLRRLLRLRL'))` | 4 |
| `print(BalancedStrings('RLLLLRRRLR'))` | 3 |

**Answer:** (penalty regime: 0 %)

Reset answer

```
def BalancedStrings(s: str) -> int:
    balance = 0
    max_splits = 0

    for char in s:
        if char == 'R':
            balance += 1
        else:
            balance -= 1

        # A balance of 0 means 'L' and 'R' count are equal
        if balance == 0:
            max_splits += 1

    return max_splits

```

| | Test | Expected | Got | |
|---|---|---|---|---|
| ✓ | `print(BalancedStrings('RLRRLLRLRL'))` | 4 | 4 | ✓ |
| ✓ | `print(BalancedStrings('RLLLLRRRLR'))` | 3 | 3 | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

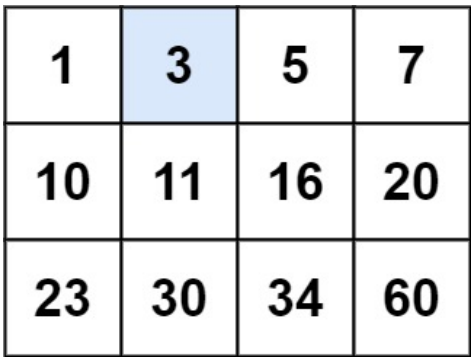
You are given an `m x n` integer matrix `matrix` with the following two properties:

- Each row is sorted in non-decreasing order.
- The first integer of each row is greater than the last integer of the previous row.

Given an integer `target`, return `True` *if* `target` *is in* `matrix` *or* `False` *otherwise*.

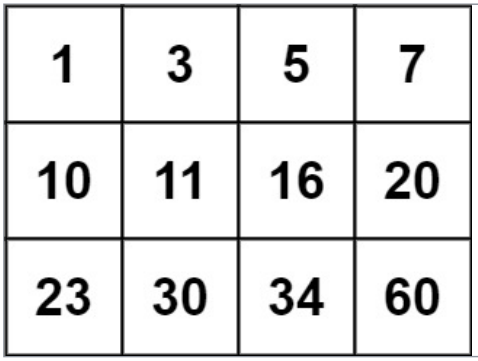
You must write a solution in `O(log(m * n))` time complexity.

**Example 1:**




```
Input: matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 3
Output: True
```

**Example 2:**




```
Input: matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 13
Output: False
```

**For example:**

| Test | Result |
|---|---|
| print(searchMatrix([[1,3,5,7],[10,11,16,20],[23,30,34,60]], 13)) | False |
| print(searchMatrix([[1,3,5,7],[10,11,16,20],[23,30,34,60]], 3)) | True |

**Answer:** (penalty regime: 0 %)

Reset answer

```
def searchMatrix(matrix: list[list[int]], target: int) -> bool:
    if not matrix or not matrix[0]:
        return False

    m, n = len(matrix), len(matrix[0])
    left, right = 0, (m * n) - 1

    while left <= right:
        mid = left + (right - left) // 2

        # Map the 1D mid index to 2D matrix coordinates
        row = mid // n
        col = mid % n
```

```
14
15          val = matrix[row][col]
16
17          if val == target:
18              return True
19          elif val < target:
20              left = mid + 1
21          else:
22              right = mid - 1
23
24      return False
25
```

| | Test | Expected | Got | |
|---|---|---|---|---|
| ✓ | print(searchMatrix([[1,3,5,7],[10,11,16,20],[23,30,34,60]], 13)) | False | False | ✓ |
| ✓ | print(searchMatrix([[1,3,5,7],[10,11,16,20],[23,30,34,60]], 3)) | True | True | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.